

10 Shoot Fish in a Barrel

The goal of this assignment is to use American Fuzzy Lop (AFL) to locate vulnerabilities in old software. I've included instructions for `openjpeg`, but if you're feeling adventurous, you can choose another target from AFL's **bug-o-rama trophy case** here:

<https://lcamtuf.coredump.cx/afl/>

If you do choose a different project, choose carefully; you want something that's not huge and not too hard to work with. If you pick a target and are having trouble working with it, consider choosing another. `Openjpeg` is pretty nice, because it is fairly simple to build and it includes a binary suitable for testing.

Give yourself plenty of time; you should really leave `afl-fuzz` running for a few hours at the very least.

Get and Build AFL

Get AFL and build it. Basically all you have to do is run `make`. I showed this in the lecture.

Get Your Target and Travel to the Past

If you're going to use `openjpeg`, clone the project from here:

<https://github.com/uclouvain/openjpeg>

Get an older version of the project. You can see available tags and their dates with this bit of magic:

```
git log --tags --simplify-by-decoration --pretty="format:%ci %d"
```

For `openjpeg`, we're going to travel back in time to 2014 like this:

```
git checkout version.2.0
```

Configure the Build

This is maybe the trickiest part. You need to (1) tell the build to use the AFL compiler `afl-gcc`, and (2) request a static build. Usually these builds make a shared library, which contains the target code, then have some small utilities that dynamically link to the shared library.

From a fuzzing standpoint, the dynamic linking can be a little confusing and introduces some uncertainty that you are fuzzing the code you want to fuzz. Fortunately, many build systems allow you to build *static* binaries, in which the library code is packaged along with the utility.

Openjpeg uses a `cmake` build. You can set it up like this:

```
mkdir build-v20-static
cd build-v20-static/
CC=~/.Applications/afl-2.52b/afl-gcc cmake .. -DBUILD_SHARED_LIBS=OFF
```

Then everything should work. Build `openjpeg`:

```
make
```

Once it's built, you've got a `bin/opj_dump` utility that we can use for fuzzing.

Fuzz

Locate or create one or more input files. I found a JPEG 2000 file here:

<https://www.fnordware.com/j2k/jp2samples.html>

Run AFL. This is the command that works for me, but you will need to adjust paths and names:

```
~/Applications/afl-2.52b/afl-fuzz -i ~/Downloads/j2k-tests/ -o afl-out --
bin/opj_dump -i @@
```

Let it go for a few hours if possible. If you were doing this “for real,” you would set it up to run for days--as long as it took until AFL decided it was done.

Reproduce Failures

This is the part that went badly for me in class. AFL finds test cases that cause crashes, so you would think you could pass those same test cases to the same target software and see the same software. It turns out it doesn't always work, as we found out.

Sometimes, the conditions under which AFL runs the target software are different from you just running it at the command line. In particular, AFL runs the target software with limited memory, which helps expose problems.

The upshot is that not all the crash test cases listed in the AFL findings directory are going to work (i.e. fail) when you try to reproduce them.

Also note that the `opj_dump` program will only accept files with a `.jp2` extension. This means you have to copy AFL's test cases to an appropriately named file before you can try to reproduce the failure. Here is an example that did reproduce the failure (a segmentation fault):

```
$ cp afl-out/crashes/id\:000016\,sig\:11\,src\:000000\,op\:havoc\,rep\:8 tc.jp2
$ bin/opj_dump -i tc.jp2

[INFO] JP2 IHDR box: compression type indicate that the file is not a conforming JP2
file (1)
Segmentation fault
```

Submission

Zip up your AFL results directory and submit it.